



PostgreSQL – Weitere Schema-Objekte

Stephan Karrer

Weitere Datenbankobjekte unter MariaDB

- **View**

Logische Sicht auf Daten in potentiell verschiedenen Tabellen

- **Sequenz**

Generierung von eindeutigen Zahlenwerten

- **Index**

Verbesserung der Zugriffsperformanz

Sichten (Views)

**Base
Table**

employees						
employee_id	last_name	job_id	manager_id	hire_date	salary	department_id
203	marvis	hr_rep	101	07-Jun-94	6500	40
204	baer	pr_rep	101	07-Jun-94	10000	70
205	higgins	ac_rep	101	07-Jun-94	12000	110
206	gietz	ac_account	205	07-Jun-94	8300	110

View

staff				
employee_id	last_name	job_id	manager_id	department_id
203	marvis	hr_rep	101	40
204	baer	pr_rep	101	70
205	higgins	ac_rep	101	110
206	gietz	ac_account	205	110

Eigenschaften von Views

- Views sind gespeicherte SELECT-Anweisungen
- Können komplexe Abfragen verkörpern
- Unterliegen als DB-Objekte eigener Zugriffskontrolle
- Views können auf Basis anderer Views erstellt werden
- DML-Anweisungen sind mit Einschränkungen möglich

Erzeugen, Ersetzen und Nutzen von Views

```
CREATE OR REPLACE VIEW emp_view AS
  SELECT last_name, salary*12 annual_salary
  FROM employees
  WHERE department_id = 20;

SELECT * FROM emp_view;
```

- REPLACE: Falls der View schon existiert, wird seine Implementierung und eventuelle Optionen ersetzt (entspricht ALTER VIEW)
- Zur Laufzeit wird die implementierende Abfrage ausgeführt und das Ergebnis der Abfrage auf dem Ergebnis der internen Abfrage geliefert
- Die implementierende Abfrage kann durchaus komplex sein

Erstellen eines komplexen Views

```
CREATE VIEW dept_sum_vu
  (name, minsal, maxsal, avgsal)
AS SELECT    d.department_name, MIN(e.salary),
             MAX(e.salary), AVG(e.salary)
  FROM        employees e JOIN departments d
  ON          (e.department_id = d.department_id)
  GROUP BY d.department_name;
```

- Komplexe Views
 - Verwendung von Funktionen und Gruppierungen
 - Verwendung mehrerer Tabellen
 - Verwendung von Unterabfragen
- DML ist auf einfachen Views üblicherweise möglich
- DML ist in der Regel nicht möglich bei komplexen Views
(<https://mariadb.com/docs/server/server-usage/views/inserting-and-updating-with-views>)

View-Anomalie

```
CREATE VIEW clerk AS
    SELECT employee_id, last_name, department_id, job_id
    FROM employees
    WHERE job_id = 'PU_CLERK'
       or job_id = 'SH_CLERK'
       or job_id = 'ST_CLERK';

UPDATE clerk SET job_id = 'PU_MAN' WHERE employee_id = 118;

CREATE OR REPLACE VIEW clerk AS
    SELECT employee_id, last_name, department_id, job_id
    FROM employees
    WHERE job_id = 'PU_CLERK' or job_id = 'SH_CLERK'
       or job_id = 'ST_CLERK'
    WITH CHECK OPTION ;
```

- Ein erfolgreich eingefügter oder veränderter Datensatz ist möglicherweise über den View nicht sichtbar!
- WITH CHECK OPTION verhindert diese Anomalie.

Ändern und Entfernen eines View

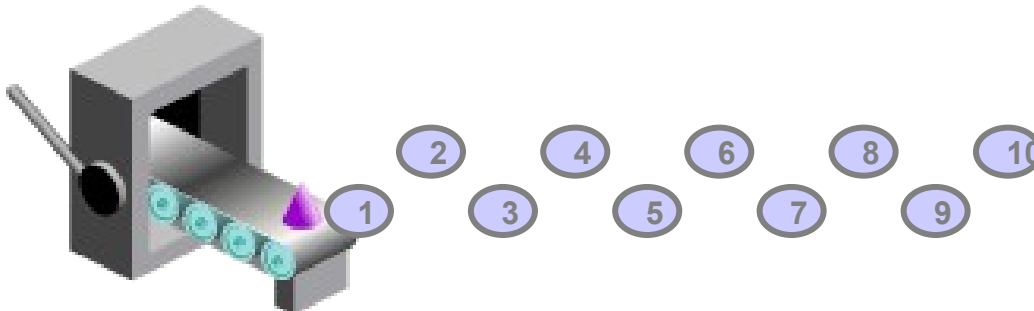
```
ALTER VIEW clerk AS
    SELECT employee_id, last_name, department_id, job_id
    FROM employees
    WHERE job_id = 'PU_CLERK';

DROP VIEW clerk;
```

- Das Löschen eines Views führt zu keinem Datenverlust, da ja die Basistabellen nach wie vor vorhanden sind.

Sequenzen

- Eigenschaften:
 - Kann automatisch eindeutige Nummern erzeugen (Ticketautomat)
 - Kann gemeinsam genutzt werden
 - Kann für die Erzeugung eindeutiger Schlüsselwerte benutzt werden
 - Die Werte können im Speicher vorgehalten (caching) werden, was den Zugriff beschleunigt



CREATE SEQUENCE: Syntax

```
CREATE [OR REPLACE] [TEMPORARY] SEQUENCE [IF NOT EXISTS] sequence_name
[AS { TINYINT | SMALLINT | MEDIUMINT | INT | BIGINT } [SIGNED | UNSIGNED]]
[ INCREMENT [ BY | = ] number ]
[ MINVALUE [=] number | NO MINVALUE | NOMINVALUE ]
[ MAXVALUE [=] number | NO MAXVALUE | NOMAXVALUE ]
[ START [ WITH | = ] number ]
[ CACHE [=] number | NOCACHE ] [ CYCLE | NOCYCLE]
```

```
-- Beispiel
CREATE SEQUENCE dept_deptid_seq
            INCREMENT BY 10
            START WITH 280
            MAXVALUE 9999
            NOCACHE
            NOCYCLE;
```

Zugriff auf die Sequenz

```
INSERT INTO departments(department_id,  
                        department_name, location_id)  
VALUES (NEXTVAL(dept_deptid_seq), 'Travel', 2500),  
      (NEXT VALUE FOR dept_deptid_seq, 'Unknown', 2500)  
  
SELECT PREVIOUS VALUE FOR dept_deptid_seq;
```

- **NEXTVAL (NEXT VALUE FOR)**
liefert den nächsten verfügbaren Sequenzwert;
jeder Aufruf verbraucht damit einen Wert.
- **PREVIOUS VALUE FOR**
Zugriff auf den aktuell geholten Wert; ist mehrfach möglich.

Ändern und Löschen einer Sequenz

```
ALTER SEQUENCE dept_deptid_seq  
        INCREMENT BY 20  
        MAXVALUE 999999  
        NOCACHE  
        NOCYCLE;
```

- Nur die zukünftige Nummernvergabe ist betroffen

```
DROP SEQUENCE dept_deptid_seq;
```

Indizes

- Ist ein Schemaobjekt
- Kann Performanzgewinn bringen
- Existiert unabhängig von der Tabelle
- Wird automatisch durch den Server benutzt und verwaltet

Erzeugung und Löschen eines Index

- MariaDB erzeugt automatisch einen Index für Primärschlüssel-Spalten bei der Tabellenerzeugung.
- Der Benutzer kann zusätzliche Indizes auf (unique und nicht-unique) Spalten erzeugen

```
CREATE INDEX emp_last_name_idx  
ON          employees(last_name);
```

```
DROP INDEX emp_last_name_idx;
```

Richtlinien für die Erzeugung eines Index

Create an index when:



A column contains a wide range of values



A column contains a large number of null values



One or more columns are frequently used together in a `WHERE` clause or a join condition



The table is large and most queries are expected to retrieve less than 2% to 4% of the rows in the table

Do not create an index when:



The columns are not often used as a condition in the query



The table is small or most queries are expected to retrieve more than 2% to 4% of the rows in the table



The table is updated frequently



The indexed columns are referenced as part of an expression